Beyond Arduino Workshop V4.0

session 1

Table of contents

01 Why beyond Arduino

02 What is Arduino?

03 Bare-metal vs HAL

04 Atmega328p Stuff

05 Bare-metal Blink!

Why beyond Arduino? Defining the Issue

- You've learned Arduino, built projects, and worked with various "Arduino-compatible" sensors and actuators. What's the next step?
- Have you ever wondered what's behind the scene of pinMode () and digitalWrite() functions?
- How do you proceed when no library exists for the module you want to use?
- What should you do if you encounter issues like library conflicts ?
- how your arduino code is compiled and flashed to the hardware ?
- Are you interested in exploring other microcontrollers (MCUs) start your embedded systems journey but unsure where to start?

What is Arduino?

Arduino is an ecosystem that consists of multiple components working together to simplify embedded system development. It includes:

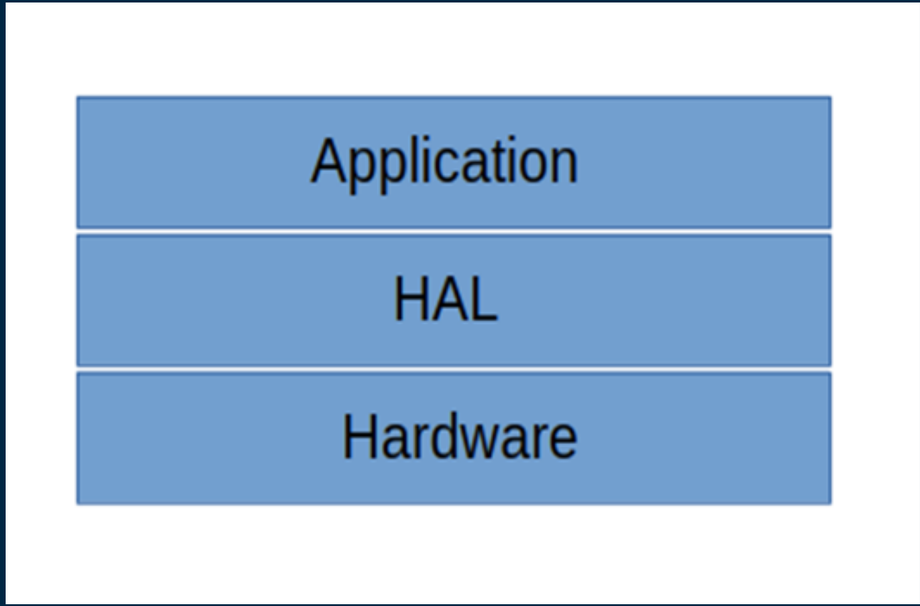
- **Development Boards** – Physical microcontroller-based boards like the Arduino Uno, Mega, Nano, and others.
- **Arduino IDE** – An integrated development environment that provides an easy-to-use interface for writing, compiling, and uploading code to Arduino boards.
- **Framework & Codebase** – The Arduino Core Libraries provide high-level functions to interact with hardware components like GPIO, PWM, I2C, SPI. They abstract away the complexity of low-level microcontroller programming.

What is HAL ?

The Hardware Abstraction Layer (HAL) provides a simplified and hardware independent interface that sits between the application code and hardware drivers, such as: GPIO drivers, Serial COM drivers, Motor drivers… etc

For Arduino, the HAL is a set of C/C++ functions/ classes and libraries that abstract away the low level code that is specific to the underlying MCU architecture.

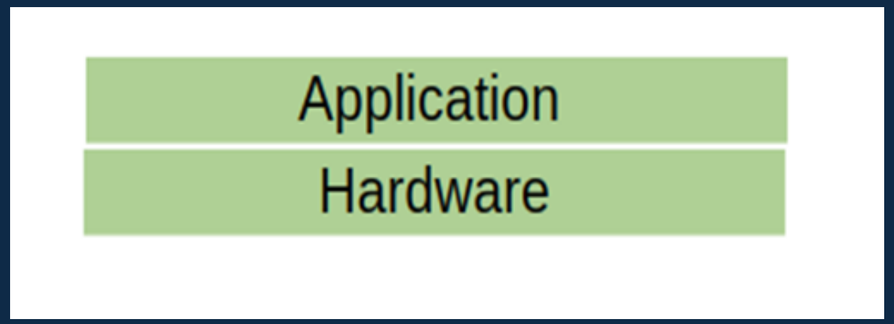
A HAL ensures portability of code across different MCU architectures, because it’s not hardware-specific code. • This means your LED blinking example can run on an UNO board and on any arduino-compatible board, regardless of the board’s microcontroller.



What is Bare-metal programming ?

Bare-metal programming refers to writing software that runs directly on the hardware without any operating system or abstraction layer.

Having full control over the microcontroller’s registers, memory, and peripherals, typically coding in low-level languages like C or assembly



The software is highly hardware-dependent, making this approach lack portability.

It also requires in-depth knowledge of hardware architectures and low-level platform details.

HAL vs Bare-metal:

to set a pin to input using arduino’s hal:

```
pinMode(13,INPUT);
```

to set a pin to input using bare-metal programming:

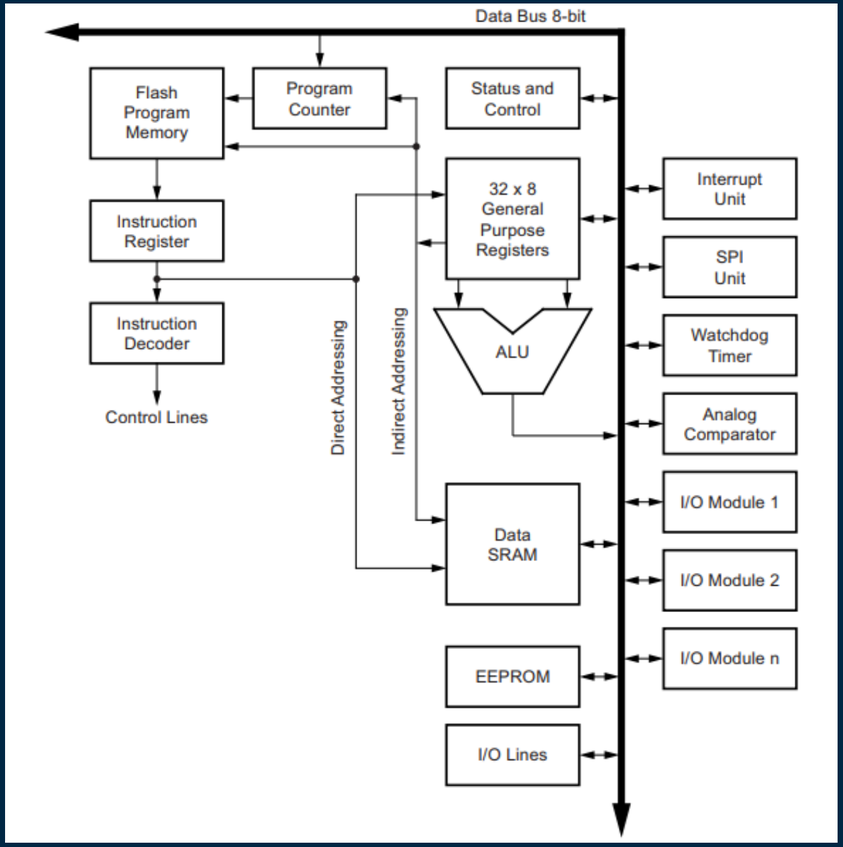
```
DDRB &= ~(1 << PB5);
```

Atmega328p stuff

The ATmega328P is an 8-bit AVR microcontroller built on the RISC Harvard architecture.

It operates at a clock speed of 16 MHz and contains 32 KB of flash memory, 2 KB of SRAM.

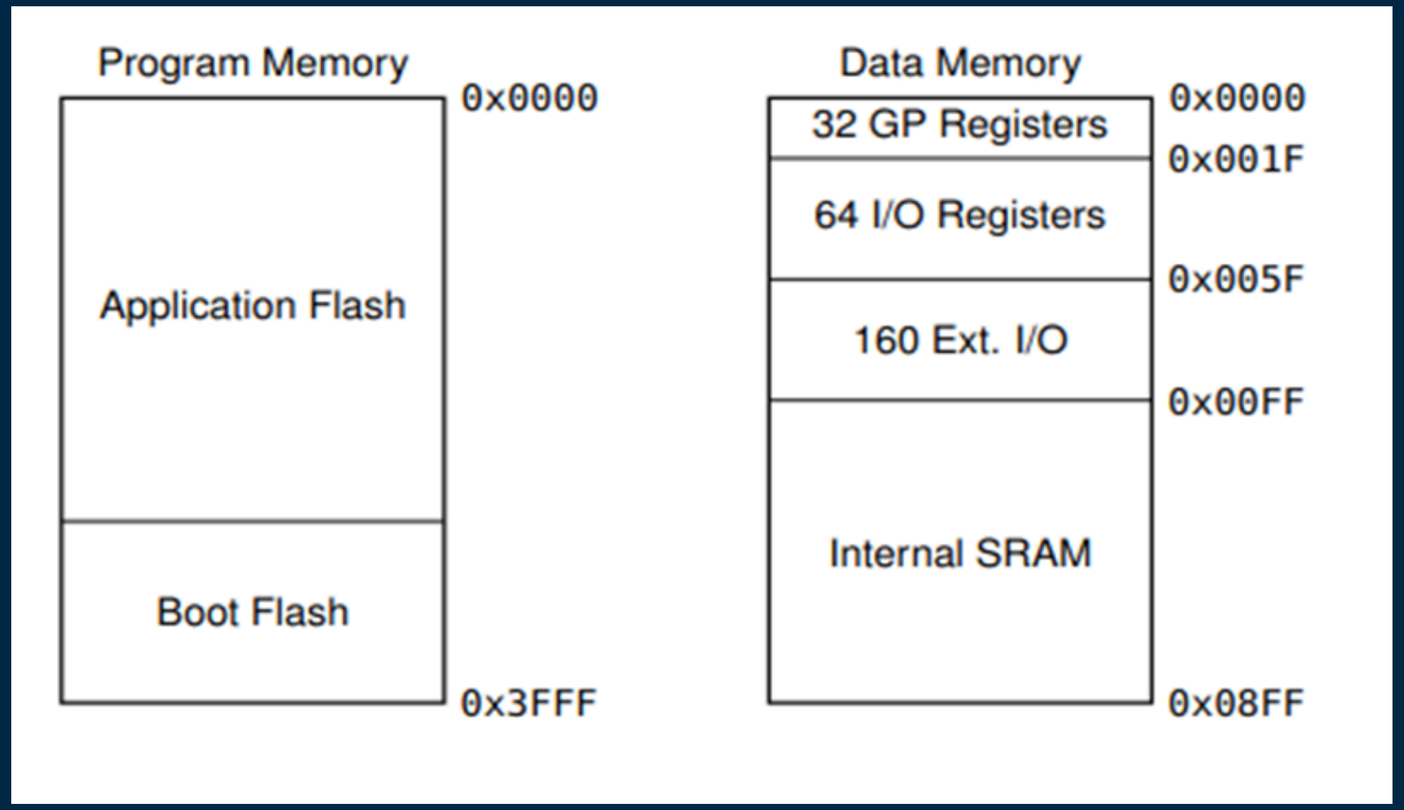
The microcontroller integrates several essential peripherals, including three timers (two 8-bit and one 16-bit), an 8-channel 10-bit ADC, a watchdog timer, and communication modules such as USART, SPI..etc



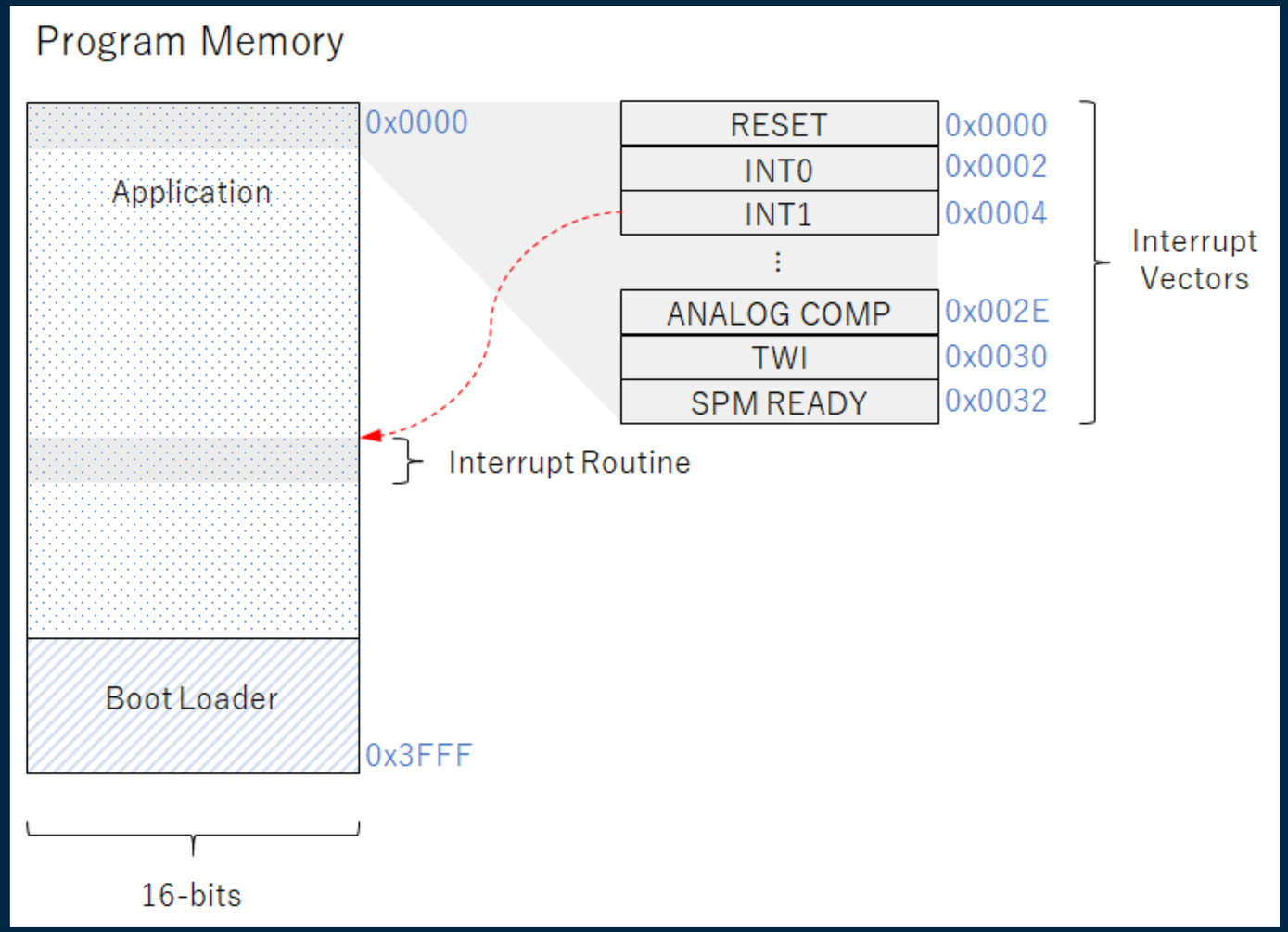
Atmega328p memory

Flash(32KB):
-> Non-volatile
-> Program memory
-> read-only

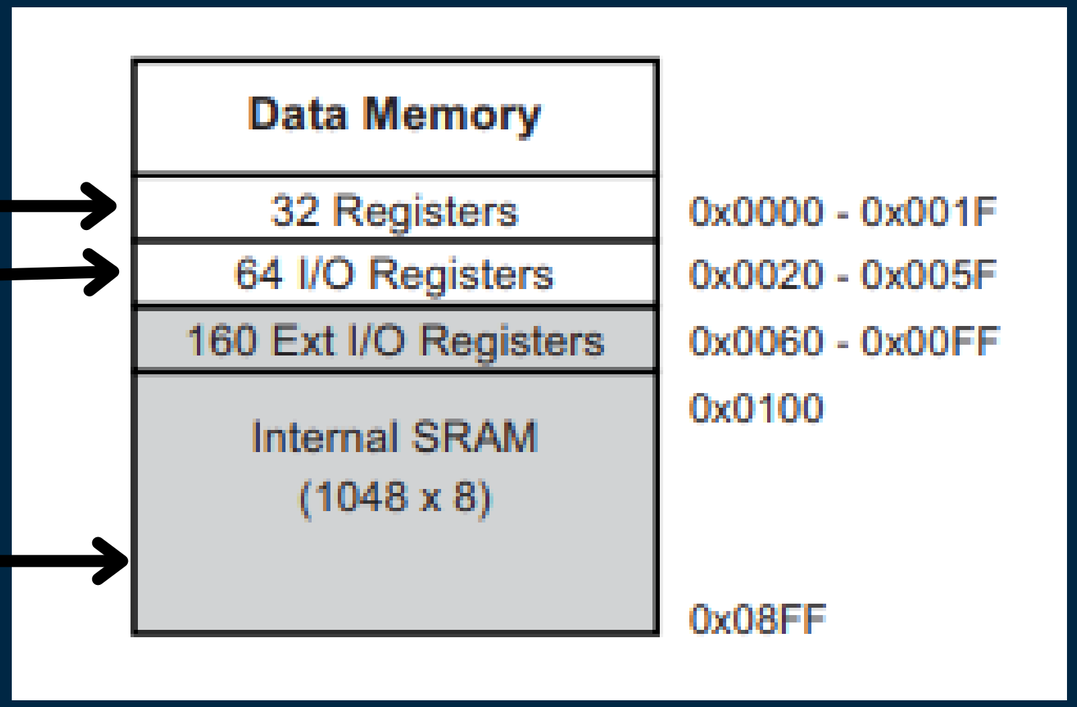
SRAM(2KB):
-> volatile
-> Data Memory - Temporary storage -> 32 8-bit core(cpu) registers reside here.



An Extended View:



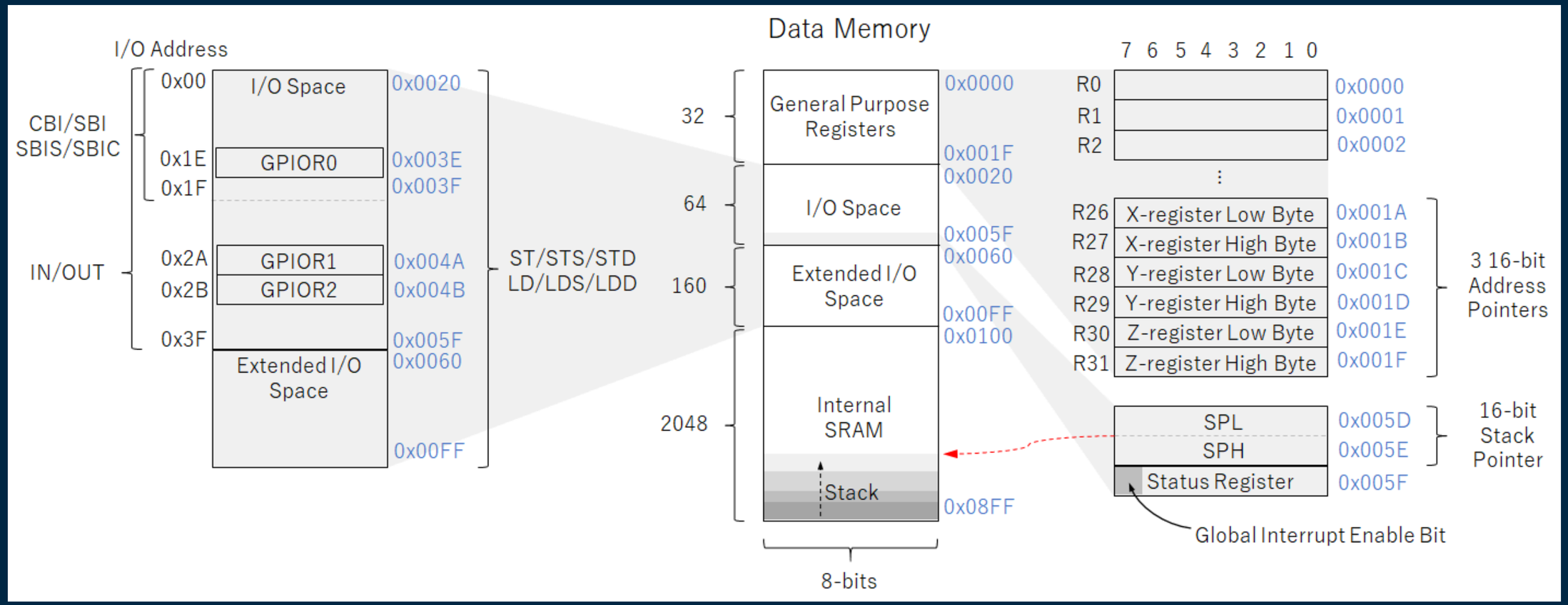
Here live 8-bit general purpose registers, that are directly accessed by the ALU. They exist physically inside the CPU core, but are addressed and included in the data memory map



Here live I/O REGISTERS! special-purpose memory locations that the CPU uses to control peripherals

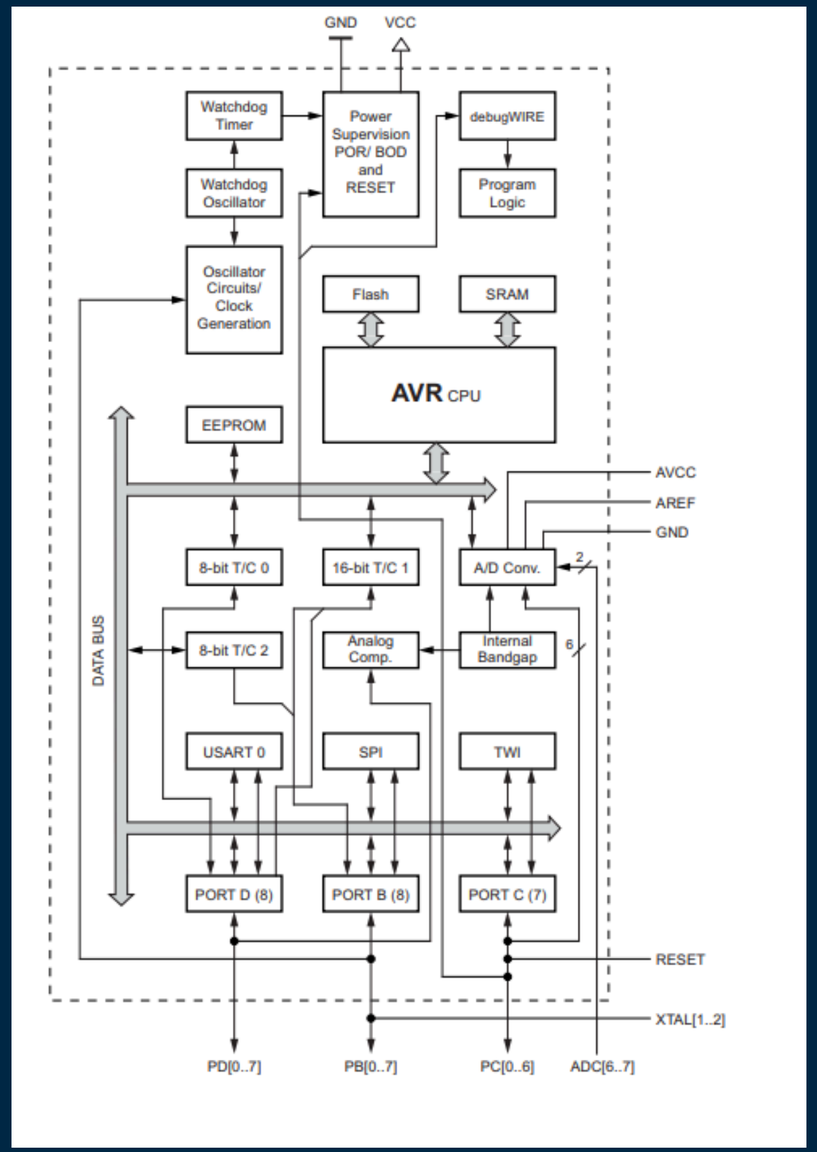
Here live the variables we create (int i = 9;), the Stack (for function calls, return addresses (PC) from interrupts & subroutines, and local vars), and other temporary data

An Extended View:



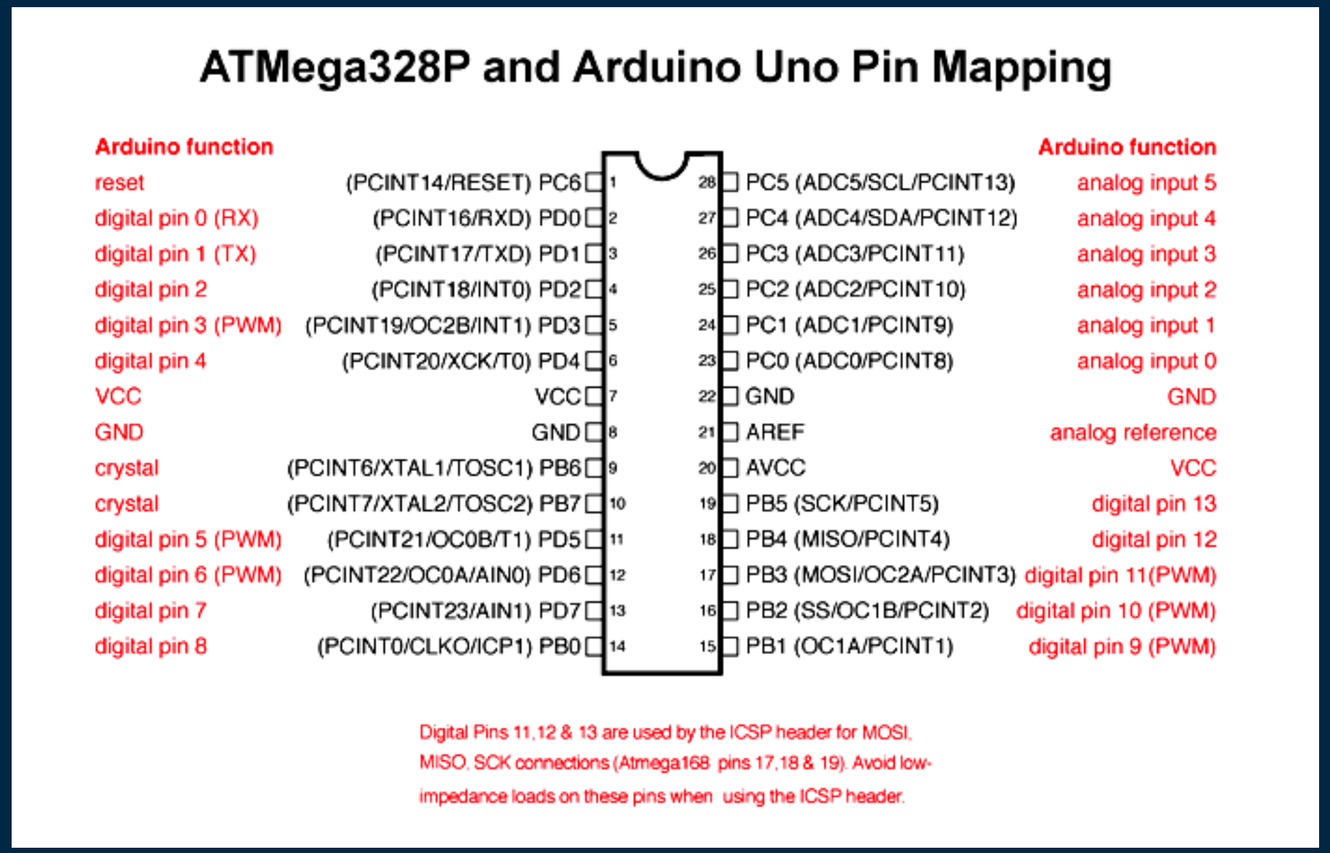
Peripheral registers

-IO registers are not like general-purpose registers.
-Each peripheral has a collection of registers, which control its behavior.
-Some registers configure settings, while others hold data.
-Configuration registers may store values or flags.
-The possibilities are endless. Read-only, write-only, write-to-clear, read-to-clear, …



Atmega328p GPIO pinout (Pins map)

In order to control arduino pins, we need to map conventional Arduino pins (0-13) to the “real” atmega328p pins. In our MCU, there are 3 ports: PORTD, PORTB, PORTC each with Pins (0-7)



GPIO registers :

Each I/O port's pin is configured and controlled using 3 register bits from 3 I/O registers: DDRx, PORTx, and PINx.

PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PINB – The Port B Input Pins Address[1]

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

DDRx: Data Direction Register (R/W), writing 0 sets pin as input, writing 1 sets pin as output.

DDRB:

0	0	1	0	0	0	0	0

= 32

DDRB5

Setting DDRB5 to 1 will set pin 13 on arduino to output

PORTx: Data Register (R/W), writing 0 sets pin value to 0 (LOW), writing 1 sets pin value to 1 (HIGH).

PORTB:

0	0	1	0	0	0	0	0

= 32

PORTB5

Setting PORTB5 to 1 will set pin 13 on Arduino high.
Setting PORTB5 to 0 will set pin 13 on arduino low .

PINx: Data Register (read-only), reading this register returns value of pins.

PINB:

0	0	1	0	0	0	0	0

= 32

PINB5

Setting PINB5 to 1 will toggle pin 13 on arduino.
Reading PINB5 will return value of pin 13 on arudino.

Always refer to the datasheet -these are not General purpose registers-

- PORT register in input mode is used to enable or disable a pull-up resistor
  Write 1 to enable the pull-up, and 0 to disable the pull-up
- PIN register in output mode can toggle the state of the corresponding PIN value
  Write 1 to toggle, 0 has no effect

Bit manipulation (bitwise C) :

1- Set a bit: we use OR for that!

There are many ways to code setting a bit to 1, but some are better than others!

1) DDRB = 32; // does the job but affects all other bits!
2) DDRB = DDRB | B00100000; // preserves bits, nice! but too many digits
3) DDRB |= 0x20; // nice :)

2- Reset a bit: we use AND for that!

1) PORTB = 0; // does the job but affects all other bits!
2) PORTB = PORTB & ~(B00100000); // preserves bits, nice! but too many digits
3) PORTB &= ~0x20; // nice :)

3- Toggling a bit: we use XOR for that!

1) PORTB = PORTB ^ B00100000; // toggles 5 th bit of PORTB
2) PORTB ^= 0x20; // nice :)

4- Using shift operator to generate mask and setting bits

There is even a BETTER way to write sth like: B00100000 and it’s by using bitwise shift operations:

We can use: (1U << 5) to represent B00100000!

```
PORTB &= ~(1U << 5);
```

We can go above and beyond and make it a MACRO* and use it as a bit mask:

```
#define PINMASK(x) (1U << x)
PORTB &= ~PINMASK(5) /*we call it a pseudo-function
```

or use the avr-libc _BV(bit) macro:

```
#define _BV(bit) (1<<bit )
```

Note: no run-time overhead when using the macros*

Bare-Metal blink (Code)

blinkled.ino

```
#define PINMASK(x) (1 << x)

void setup(){

  DDRB |= PINMASK(5); /* pinMode(13, OUTPUT); */
}

void smalldelay(){
  for (uint32_t volatile i =0 ; i<0x000FFFFF ; i++);
}

void loop()

  PORTB |= PINMASK(5); /* digitalWrite(13, HIGH) */
  smalldelay();
  PORTB &= ~PINMASK(5);
  smalldelay();
}
```

comparison:

- **program size:**

Using Arduino HAL:

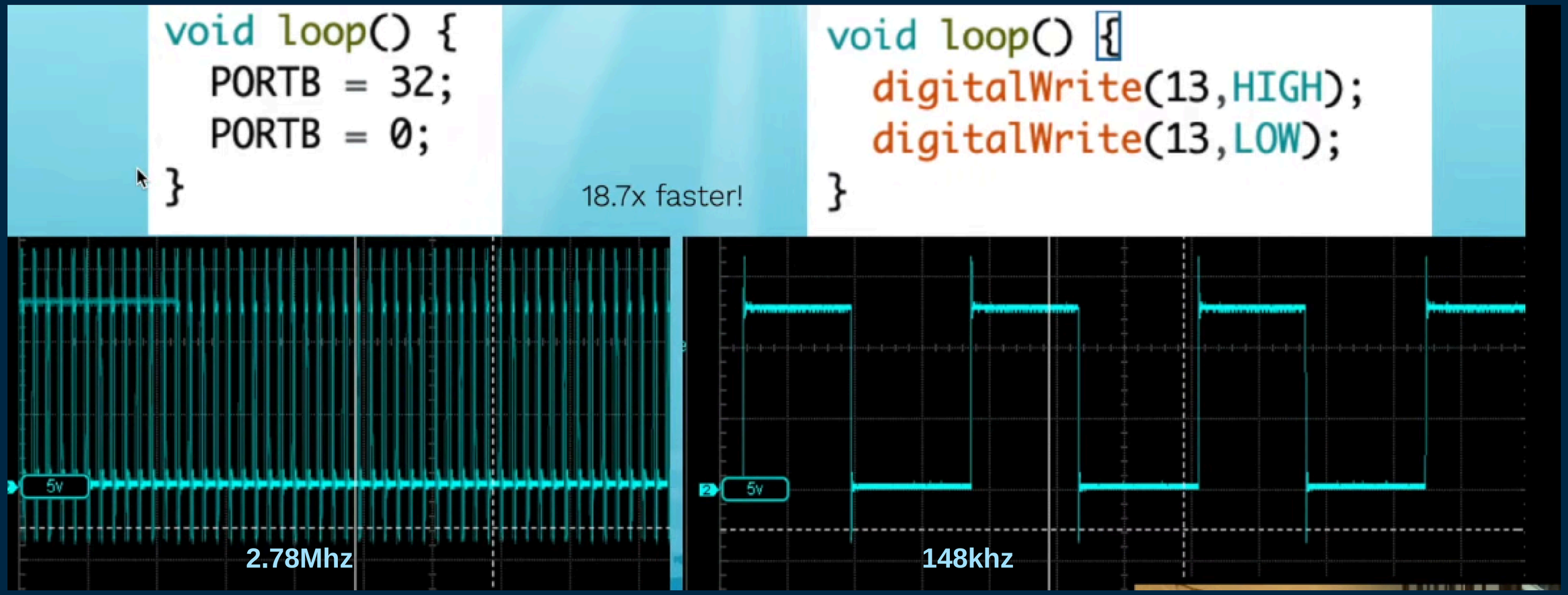
```
Sketch uses 924 bytes (2%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.
```

Using Bare-metal:

```
Sketch uses 534 bytes (1%) of program storage space. Maximum is 32256 bytes.
Global variables use 9 bytes (0%) of dynamic memory, leaving 2039 bytes for local variables. Maximum is 2048 bytes.
```

comparison:

you can see the HAL overhead !



from Mitch Davis - Fundamentals of microcontrollers

THANK YOU!